Srishyla Educational Trust (R)

GM UNIVERSITY

(Established under the Karnataka State Act No. 19 of 2023)
P. B. Road, Davanagere, Karnataka - 577 006
Toll Free No.: 18001237099, Ph: +91 93640-99720 /721
E-mail: info@gmu.ac.in, Website: www.gmu.ac.in

Object Oriented Programming

(Course Code: UE24CS2304)

Lab Manual

For III Semester

VISION & MISSION OF THE GMU

VISION

GM University will have a transformative impact on society through continual innovation in education, research, skill development, creativity, and entrepreneurship.

MISSION

- To disseminate knowledge and conduct research in academic areas such as science, engineering, commerce, management, health, humanities & social sciences, and legal studies with learner centric approach.
- To teach skills such as critical thinking, creativity and innovation, collaboration, communication, technical and digital, flexibility and adaptability, cultural values, and leadership and responsibility
- To develop global citizens by educating students on emotional, physical, social, economic, environmental, spiritual dimensions of human growth in addition to intellectual pursuits
- To address real-world challenges and to establish the groundwork for entrepreneurship and lifelong learning.

OOPS Laboratory Problem Statements

1. Develop a JAVA program to add TWO matrices of suitable order N (The value of N should be read from command line arguments).

2. A class called Employee, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method raiseSalary (percent) increases the salary by the given percentage. Develop the Employee class and suitable main method for demonstration.

3. Develop a JAVA program to create a class named shape. Create three sub classes namely: circle, triangle and square, each class has two member functions named draw () and erase (). Demonstrate polymorphism concepts by developing suitable methods, defining member data and main program**.**

4. Develop a JAVA program to create a package named mypack and import & implement it in a suitable class.

5. Write a Java program to read two integers a and b. Compute a/b and print, when b is not zero. Raise an exception when b is equal to zero.

6. Write a java program in which you will declare an abstract class vehicle inherits this class from two classes car and truck using the method engine in both display “car has good engine” and “truck has good engine”.

7. Write a program to illustrate creation of threads using runnable class. (start method start each of the newly created thread. Inside the run method there is sleep() for suspend the thread for 500 milliseconds).

8. Design and implement a Java program using the Swing framework to:
   a) Create a text box to accept user input.
   b) Add a submit button to the interface.
   c) Display the user-provided input upon clicking the submit button.

ECLIPSE IDE INSTALLATION

1. Download Eclipse IDE

1. **Visit the Eclipse Download Page**:
   - Go to the Eclipse Downloads page.
2. **Select the Eclipse IDE for Java Developers**:
   - Choose the version suited for Java development. You'll typically see options like "Eclipse IDE for Java Developers" or "Eclipse IDE for Java EE Developers." Click on the "Download" button for the one that suits your needs.
3. **Download the Installer**:
   - Depending on your operating system (Windows, macOS, or Linux), download the appropriate installer or archive file.

2. Install Eclipse IDE

1. **Run the Installer**:
   - Double-click the downloaded .exe file to launch the Eclipse Installer.
2. **Choose Your IDE**:
   - In the installer window, select "Eclipse IDE for Java Developers" or any other relevant package.
3. **Select Installation Location**:
   - Choose the directory where you want Eclipse to be installed. The default is usually fine, but you can change it if needed.
4. **Start Installation**:
   - Click "Install" and accept any license agreements.
5. **Launch Eclipse**:
   - Once installation is complete, you can start Eclipse from the installer or find it in your start menu.

3. Set Up Eclipse

1. **Select a Workspace**:
   - When you first launch Eclipse, it will ask you to select a workspace. This is the directory where your projects and files will be stored. You can use the default or choose a different location.
2. **Configure Preferences (Optional)**:
   - Go to Window > Preferences (or Eclipse > Preferences on macOS) to adjust various settings according to your needs.

3. **Install Java Development Kit (JDK)**:
   - Ensure you have the JDK installed on your system. Eclipse requires the JDK to compile and run Java programs. You can download it from Oracle's website or use OpenJDK.

4. **Verify Java Setup**:
   - Go to Window > Preferences > Java > Installed JREs to make sure your JDK is properly configured. If not, you can add it manually here.
5. **Create a New Java Project**:
   - Go to File > New > Java Project to start a new Java project. Follow the prompts to set up your project.

Lab Program 1: Develop a JAVA program to add TWO matrices of suitable order N (The value Of N should be read from command line arguments)

Arrays in Java : An array is a fundamental data structure used in programming to store a collection of elements. Each element in an array is identified by an index or a key, and all elements are of the same type. Arrays are widely used due to their simplicity and efficiency in accessing and managing data.

1. **One-Dimensional Arrays:**
   - A one-dimensional array is a list of elements stored in contiguous memory locations.
   - Declaration: int[] array;
   - Initialization: array = new int[10]; // An array of 10 integers.
2. **Two-Dimensional Arrays:**
   - A two-dimensional array can be thought of as an array of arrays, or a matrix. It is essentially a grid of elements.
   - Declaration: int[][] matrix;
   - Initialization: matrix = new int[3][3]; // A 3x3 matrix (or 2D array) of integers.

```
import java.util.Scanner;

public class Addmatrix
{
public static void main(String[ ] args)
 {
int p, q, m, n;
 Scanner s = new Scanner(System.in);
System.out.print("Enter number of rows in first
matrix:"); p = s.nextInt();
System.out.print("Enter number of columns in first
 matrix:"); q = s.nextInt();
System.out.print("Enter number of rows in second
matrix:"); m = s.nextInt();
 System.out.print("Enter number of columns in second
 matrix:"); n = s.nextInt();
 if (p == m && q == n)
{
   int a[][] = new int[p][q];
   int b[][] = new int[m][n];
   int c[][] = new int[m][n];
   System.out.println("Enter all the elements of first matrix:");
   for (int i = 0; i < p; i++)
  {
  for (int j = 0; j < q; j++)
```

```
        {
            a[i][j] = s.nextInt();
        }
    }
    System.out.println("Enter all the elements of second matrix:");
  for (int i = 0; i < m; i++)
   {
      for (int j = 0; j < n; j++)
     {
        b[i][j] = s.nextInt();
      }
}
System.out.println("First Matrix:");
for (int i = 0; i < p; i++)
 {
      for (int j = 0; j < q; j++)
     {
         System.out.print(a[i][j]+" ");
       }
      System.out.println("");
   }
   System.out.println("Second Matrix:");
   for (int i = 0; i < m; i++)
    {
       for (int j = 0; j < n; j++)
       {
           System.out.print(b[i][j]+" ");
        }
      System.out.println("");
    }
   for (int i = 0; i < p; i++)
   {
      for (int j = 0; j < n; j++)
       {
         c[i][j] = a[i][j] + b[i][j];
       }
   }
   System.out.println("Matrix after addition:");
   for (int i = 0; i < p; i++)
  {
       for (int j = 0; j < n; j++)
      {
```

```
                System.out.print(c[i][j]+" ");
            }
            System.out.println("");
        }
    }
    {
        System.out.println("Addition would not be possible");
    }

    }


}
```

OUTPUT:

1. Enter number of rows in first matrix:2 Enter number of columns in first matrix:2 Enter number of rows in second matrix:2
   Enter number of columns in second matrix:2 Enter all the elements of first matrix:
   10
   12
   20
   22
   Enter all the elements of second matrix: 10
   12
   20
   22
   First Matrix:
   10 12
   20 22
   Second Matrix:
   10 12
   20 22
   Matrix after addition:

20 24
40 44

2. Enter number of rows in first matrix:2 Enter number of columns in first matrix:3 Enter number of rows in second matrix:3
Enter number of columns in second matrix:2 Addition would not be possible

Lab Program 2: A class called Employee, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method raiseSalary (percent) increases the salary by the given percentage. Develop the Employee classand suitable main method for demonstration

Class:

A class is a blueprint or template for creating objects. It defines a data structure that contains fields (attributes) and methods (functions or behaviors) that operate on the data. A class encapsulates data for the object and methods to manipulate that data.

Object:

An object is an instance of a class. It represents a concrete realization of the class with actual values for the fields defined by the class. Each object has its own state and can use the methods defined in the class.

Method:

In Java, a **method** is a block of code that performs a specific task. Methods are used to define the behavior of objects and can be invoked to execute that behavior. Methods can be thought of as functions that belong to a class and can operate on the data within that class.

```
import
java.util.*;
class
employee
{
  float per=0;
  double salary=0,isalary=0;
  Scanner s = new
  Scanner(System.in); void
```

```
 raisesalary()
   {
    System.out.println("Enter the Employee
    Salary"); salary=s.nextDouble();
    System.out.println("Enter the Percentage of salary to be
    incremented"); per=s.nextFloat();
    isalary=salary+((salary*per)/100);
    }
 }
public class salaryhike
{
public static void main(String[] args)
 {
                // TODO Auto-generated method stub int eid=0;
   String name;
Scanner s = new Scanner(System.in);
System.out.println("Employee increment\n");
System.out.println("Enter the Employee ID\n");
eid=s.nextInt();
 System.out.println("Enter the Employee
Name\n"); name=s.next();
 employee e=new employee();
e.raisesalary();
System.out.println("Employee
name:"+name);
System.out.println("Employee ID:"+eid);
System.out.println("Present
salary:"+e.salary);
System.out.println("Incremented
salary:"+e.isalary); s.close();
}
}
```

OUTPUT:

```
Employee
increment Enter
the Employee ID
123
Enter the Employee
Name Deepa
```

```
Enter the Employee
Salary 45000
Enter the Percentage of salary to be
incremented 15
Employee name:
Deepa
Employee ID:123
Present
salary:45000.0
Incremented salary:51750.0
```

Lab Program 3: Develop a JAVA program to create a class named shape. Create three sub classes namely: circle, triangle and square, each class has two member functions named draw () and erase (). Demonstrate polymorphism concepts by developing suitable methods, defining member data and main program.

Polymorphism:

Polymorphism in Java allows objects of different classes to be treated as objects of a common superclass. This is primarily achieved through:

1. **Compile-Time Polymorphism:** Also known as method overloading.
2. **Run-Time Polymorphism:** Also known as method overriding.

Demonstration of Polymorphism

We'll create a Java program to illustrate these concepts:

1.Compile-Time Polymorphism (Method Overloading)

Method overloading allows multiple methods with the same name but different parameters to exist within the same class. The method to be executed is determined at compile time based on the method signature.

2.Run-Time Polymorphism (Method Overriding)

Method overriding allows a subclass to provide a specific implementation of a method that is already defined in its superclass. The method to be executed is determined at runtime based on the actual object type, not the reference type.

```
class Shape
{
 protected String
name; public
Shape(String
name)
{
```

```
 this.name = name;
 }
 public void draw()
 {
System.out.println("Drawing a " + name);
 }
 public void erase()
 {
 System.out.println("Erasing a " + name);
 }
 }
 class Circle extends Shape
 {
 private double radius;
public Circle(String name, double radius)
 {
 super(name
 );
 this.radius =
 radius;
 }
 @Overrid
public void
draw()
{
 System.out.println("Drawing a circle with radius " + radius);
 }
 @Overrid
public void
erase()
 {
System.out.println("Erasing a circle with radius " + radius);
}
 }

 class Triangle extends Shape
 {
 private double base;
private double height;
```

```
 public Triangle(String name, double base, double height)
{
super(name);
 this.base =
base;
this.height =
height;
 }
 @Overrid
public void
draw()
{
System.out.println("Drawing a triangle with base " + base + " and height " + height);
 }
 @Overrid
public void
erase()
{
 System.out.println("Erasing a triangle with base " + base + " and height " + height);
 }
}
 class Square extends Shape
{
 private double side;
 public Square(String name, double side)
 {
super(name
);
this.
Side=side;
 }
 @Overrid
public void
draw()
{
 System.out.println("Drawing a square with side length " + side);
}
 @Overrid
e
```

```
public void
erase()
{
System.out.println("Erasing a square with side length " + side);
}
}
public class ShapeDemo
{
public static void main(String[] args)
{
  Shape[] shapes = new Shape[3];
  shapes[0] = new Circle("Circle", 5.0);
  shapes[1] = new Triangle("Triangle",
  4.0, 6.0); shapes[2] = new
  Square("Square", 3.0);
  for (Shape shape : shapes)
    {
        shape.draw();
        shape.erase();
        System.out.prin
        tln();
     }
  }
}
```

OUTPUT:

```
Drawing a circle with radius
5.0 Erasing a circle with
radius 5.0
Drawing a triangle with base 4.0 and height
6.0 Erasing a triangle with base 4.0 and height
6.0 Drawing a square with side length 3.0
Erasing a square with side length 3.0
```

Lab Program 4:Develop a JAVA program to create a package named mypack and import & implement it in a suitable class.

Package:

Packages in Java are a namespace mechanism that organizes classes and interfaces into a hierarchical structure. They help in grouping related classes and interfaces together, making the code more modular and easier to manage.

```
package mypack;

public class MyClass
 {
public void displayMessage()
{
System.out.println("Hello from mypack.MyClass!");
 }
}

 import mypack.MyClass;
 public class Main
{
public static void main(String[] args)
{
 MyClass myObj = new MyClass();
 myObj.displayMessage();
}
 }
```

OUTPUT:

Hello from mypack.MyClass!

Lab Program 5: Write a program to illustrate creation of threads using runnable class. (start method start each of the newly created thread. Inside the run method there is sleep() for suspend the thread for 500 milliseconds).

What is a Thread?

A thread is a lightweight process or a single path of execution in a program. Multiple threads can run in parallel, sharing the same resources such as memory but operating independently.

```
class MyRunnable implements Runnable
{
@Override
 public void run()
 {
try {
                    // Sleep for 500 milliseconds (0.5 seconds)
 Thread.sleep(500);
 System.out.println(Thread.currentThread().getName() + " is
 running.");
       }
 catch (InterruptedException e)
  {
     System.err.println(Thread.currentThread().getName() + " was interrupted.");
   }
  }
}

public class ThreadCreationUsingRunnable
{
 public static void main(String[] args)
{
        // Create two instances of the MyRunnable class

MyRunnable myRunnable1 = new MyRunnable();
MyRunnable myRunnable2 = new MyRunnable();

                        // Create two threads and pass the runnable instances

to them Thread thread1 = new Thread(myRunnable1, "Thread 1");
Thread thread2 = new Thread(myRunnable2, "Thread 2");
```

```
    // Start both threads
  thread1.start();
  thread2.start();
 }
}
```

OUTPUT:

Thread 1 is
running.
Thread 2 is
running.

Lab Program 6: Write a java program in which you will declare an abstract class vehicle inherits this class from two classes car and truck using the method engine in both display "car has good engine" and "truck has good engine".

```
package p6;
abstract class vehicle
{
        abstract void engine();
}
class car extends vehicle
{
        public void engine()
        {
                System.out.println("Car has good engine");
        }
}
class truck extends vehicle
{
        public void engine()
        {
                System.out.println("Truck has good engine");
        }
}
public class TestVehicle
 {
        public static void main(String[] args)
```

```
{
        vehicle v=new car();
        v.engine();
        vehicle n=new truck();
        n.engine();
    }
```

OUTPUT:

```
Car has good
engine Truck
has good engine
```

Lab Program 6: Write a Java program to read two integers a and b. Compute a/b and print, when b is not zero. Raise an exception when b is equal to zero.

What is an Exception? An exception is an event that disrupts the normal flow of a program's execution. It represents errors such as invalid input, missing files, or division by zero.

```
import java.util.Scanner;
class division
{
  public static void main(String[] args)
   {
      int a,b,result;
      Scanner S = new Scanner(System.in);
      System.out.println("Enter two integers");
      a=S.nextInt();
      b=S.nextInt();
      try

      {
        result=a/b;
        System.out.println("Result="+resu
        lt);
        }
       catch(ArithmeticException e)
      {
        System.out.println("Exception caught:Divide by zero error"+e);
       }
       S.close();
    }
```

```
}
```

OUTPUT:

1. Enter two
   integers
   45
   5
   Result=9
2. Enter two
   integers
   34
   0
   Exception caught:Divide by zero errorjava.lang.ArithmeticException: /by Zero

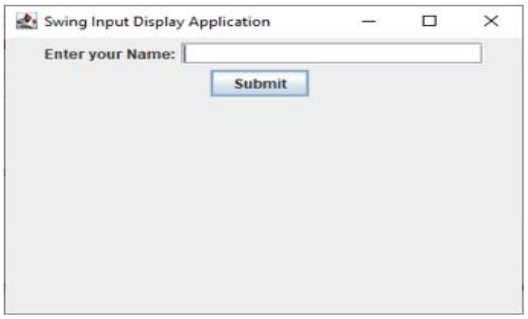
Lab Program 8: Design and implement a Java program using the Swing framework to:

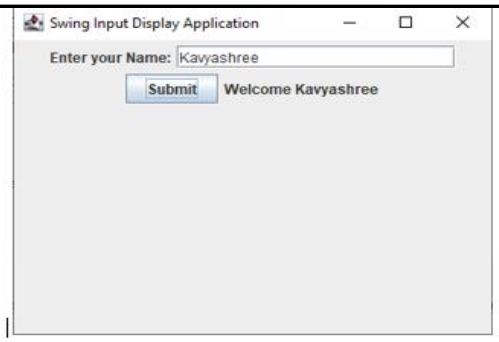
1. **Create a text box to accept user input.**
2. **Add a submit button to the interface.**
3. **Display the user-provided input upon clicking the submit button.**

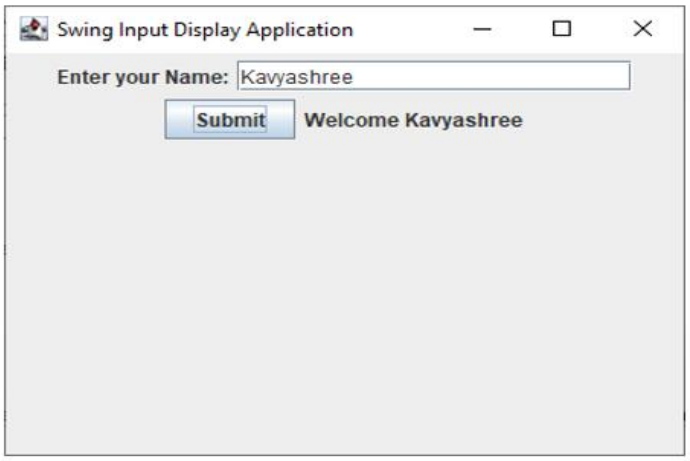
```
Import
javax.swing.*;
import
java.awt.*;
import
java.awt.event.*
;

public class myswing {
   public static void main(String[] args) {
      // Create the main JFrame
      JFrame frame = new JFrame("Swing Input Display Application");
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      frame.setSize(400, 300);
      frame.setLayout(new FlowLayout());

      // Create a JLabel to display instructions
      JLabel instructionLabel = new JLabel("Enter your Name:");
      frame.add(instructionLabel);

      // Create a JTextField for user input
      JTextField textField = new
      JTextField(20); frame.add(textField);
```

```
        // Create a JButton for submission
        JButton submitButton = new JButton("Submit");
        frame.add(submitButton);

        // Create a JLabel to display the
        result JLabel resultLabel = new
        JLabel(); frame.add(resultLabel);

        // Add ActionListener to the submit button
        submitButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Get the input from the text
                field String userInput =
                textField.getText();

                // Display the input in the result label
                resultLabel.setText("Welcome"+ " " + userInput);
    }
});

        // Make the frame visible
        frame.setVisible(true);
      }
  }
```

OUTPUT:


Swing Input Display Application
Enter your Name:
Submit


Swing Input Display Application
Enter your Name:
Kavyashree
Submit
Welcome Kavyashree


Swing Input Display Application
Enter your Name:
Kavyashree
Submit
Welcome Kavyashree